Exercise 1 — Book Management

Book class

- Private fields: `isbn` (String), `title` (String), `author` (String), `yearPublished` (int), `price` (double).
- A no-argument constructor, a constructor that sets all five fields, and getters/setters for every field.
- `display()`: prints all five fields to the console in a readable, labeled format (not just a raw concatenation).

Storage

- Store the books in an `ArrayList<Book>` rather than a plain array — the menu needs to add books one at a time with no fixed upper limit, which a fixed-size array can't do cleanly.

Menu (shown repeatedly until the user exits):

```
===== Book Management Menu =====
1. Add new Book
2. Display all books
3. Search book by ISBN
4. Search book by name
5. Exit
Choose an option:
```

Menu behaviour

- Loop showing the menu and reading a choice; on anything other than 1–5, print an error and show the menu again. Stop looping once 5 is chosen.
- 1 — Add new Book: prompt for all five fields from the keyboard and add the new `Book` to the list. Reject (and re-prompt for) an `isbn` that already belongs to a book already in the list — ISBN is the value Option 3 searches by, so it needs to actually identify one book.
- 2 — Display all books: call `display()` on every book in the list, in the order they were added.
- 3 — Search by ISBN: prompt for an ISBN, do an exact match against every book's `isbn`, and print the matching book's `display()` output, or a "No book found" message if there's no match.
- 4 — Search by name: prompt for a search term, and match it against every book's title with a case-insensitive substring check (not an exact match — title search is more useful when it doesn't require the full, exact title) — print every match, or a "No book found" message if there are none.
- 5 — Exit: end the program.

Exercise 2 — Static Arrays

Setup

- A static `int[10]` array, with all 10 elements assigned a value.
- Since a plain Java array has a fixed length, "add to the end if there's room" and "remove at position i" only make sense if you track how many of the 10 slots are actually in use — keep a separate `int size` (starting at 10, or less if you want room to demonstrate adding) alongside the array. Insert/remove/add operations shift elements within the existing 10 slots and update `size`; they never resize the array itself.

Operations to implement

- Print all elements currently in use (indices 0 through size−1).
- Find the maximum and the minimum value among the elements in use.
- Calculate the sum and the average of the elements in use.
- Add an element to the end: only if `size < 10`; otherwise report that the array is full and do nothing.

- Remove the element at a given position i: shift every later element one slot left, then decrement `size`.
- Insert an element x at position i: shift every element from i onward one slot right (dropping the last one if `size` is already 10), place x at position i, and increment `size` (unless it was already full, in which case the array stays full and the last element is discarded to make room).
- Linear search for an element x: scan from index 0 to size–1 and report the first matching index, or report "not found".
- Rotate left/right by k positions: a left rotation moves each element k positions toward index 0, wrapping the ones that fall off the front around to the end (and the mirror image for a right rotation). Take k from the keyboard; reduce it modulo `size` first so a k larger than the array still behaves correctly.
- Reverse the elements in use, in place.
- Check whether the elements in use read the same forwards and backwards (palindrome check).
- Sort the elements in use in ascending order — implement this twice, once with bubble sort and once with selection sort, as two separate methods.

Exercise 3 — Dynamic Arrays (ArrayList)

Setup

- An `ArrayList<Integer>` with 5 integers added to it to start.

Operations to implement

- Print all elements.
- Access the element at a given position with `get(index)`.
- Add an element at the beginning (`add(0, x)`), at the end (`add(x)`), or at a specific position (`add(index, x)`).
- Remove an element by position vs. by value: `ArrayList<Integer>.remove()` is overloaded, and this is a common trap — `remove(int index)` removes by position, while `remove(Object o)` removes by value. Because the list holds boxed `Integer`s, calling `list.remove(5)` removes the element at index 5, **not** the value 5. To remove by value you must force the Object overload, e.g. `list.remove(Integer.valueOf(5))`. Implement both as clearly separate operations so the distinction is unmissable.
- Check whether an element exists with `contains()`.
- Count how many times a given element appears (a manual scan, or `Collections.frequency()`).
- Filter the even elements into one new ArrayList and the odd elements into another.
- Merge two ArrayLists into one new list containing all elements of both (e.g. via `addAll()` on a copy, so neither original list is mutated).
- Find the most frequently occurring element. If more than one value ties for the highest frequency, report whichever of them was encountered first while scanning the list.
- Remove duplicate elements, keeping the first occurrence of each value and preserving the original order of what remains.

Exercise 4 — Array Statistics

Input

- Read from the keyboard: the number of elements, then that many integers, then a threshold integer.

Output

- The average of all elements (as a decimal, not truncated to an int).

- The second largest element in the array.
- The count of elements strictly greater than the threshold.

"Second largest" — resolving the ambiguous case

- "Second largest" means the second largest *distinct* value, not just the second position after sorting. For `[5, 5, 3]`, the second largest is `3`, not another `5` — find the maximum first, then find the maximum among only the elements strictly less than that.
- If every element in the array is equal (so there is no value strictly less than the maximum), or the array has fewer than 2 elements, there is no second largest — print a message saying so instead of a number.

Exercise 5 — Product & Sales Management

Product class

- Fields: `id` (Integer), `name` (String), `price` (Float), `quantity` (Integer) — quantity is the current stock on hand.

Order class

- Fields: `id` (Integer), `quantity` (Integer), `date` (LocalDateTime).
- `id` here is the **product's** id — which product this order was for — not a separate unique order number; the class only needs to remember what was sold, how much, and when. Name the field or its accessor clearly (e.g. a getter called `getProductId()`) so this isn't confused with a record's own identity later.

Encapsulation

- All fields on both Product and Order should be private, with public getters and setters for each — nothing outside the class should read or write a field directly.

Menu (shown once when the program starts, then again after every option):

```
Product Management Program
1. Enter Data | 2. Low-Stock Report | 3. Sort by Price
4. Sell Product | 5. Sales Report | 6. Exit
```

Option 1 — Enter Data

- Ask for the total number of products to enter, then collect each one's full information from the keyboard.
- Validate: `0 <= price <= 10,000,000` and `0 <= quantity`; re-prompt on an invalid value rather than discarding the whole entry.
- Reject (and re-prompt for) a product id that duplicates one already entered — Option 4 looks products up by id, so ids need to actually be unique.

Option 2 — Low-Stock Report

- Show the 3 products with the lowest stock quantity, sorted smallest to largest. If fewer than 3 products have been entered, show all of them.

Option 3 — Sort by Price

- Display every product, sorted by price from highest to lowest.

Option 4 — Sell Product

- Display the current product list, then prompt for a product id and a quantity to sell.
- The id must match an existing product; the quantity must not exceed that product's current stock — re-prompt (or report an error) if either check fails.

- On success: subtract the sold quantity from that product's stock, and record the sale as a new `Order` (product id, quantity sold, current date/time) in a running sales history list — this history is what Option 5 displays, so the sale has to be saved somewhere durable for the rest of the program's run, not just applied to the stock count and forgotten.

Option 5 — Sales Report

- Display every recorded order: product id, quantity sold, and date/time. Also look up and show the product's name next to each entry — a bare product id isn't very readable on its own, and the lookup is simple since the product list is already in memory.

Option 6

- Exit the program.

Exercise 6 — Electric Bill Management

Customer class

- Fields: `headOfHousehold` (String), `houseNumber` (String or int), `meterId` (String) — private, with getters/setters.

Bill class

- Fields: a reference to the `Customer` the bill belongs to (not just a copy of the house number — holding the actual object avoids the two getting out of sync), `previousReading` (int), `newReading` (int), `amountDue` (double).

Household management methods

- Since Customer is just a data holder, put the add/remove/edit operations in a separate manager class (e.g. `CustomerManager`) that holds a `List<Customer>`:
- Add: append a new Customer to the list.
- Remove: delete a Customer from the list, identified by meter id (the one field that should be unique per household).
- Edit: look up a Customer by meter id and update its other fields.

Bill calculation

- A method on `Bill` (e.g. `calculateAmountDue()`) that computes `(newReading - previousReading) × 5`, stores the result into `amountDue`, and also returns it.

Exercise 7 — Library Borrowing Card Management

Student class

- Fields: `fullName` (String), `age` (int), and the student's class/section (String) — name that field something like `className` or `classSection`, not `class`: `class` is a reserved keyword in Java and can't be used as a field name at all.

BorrowCard class

- Fields: `slipId` (String or int), `borrowDate` (LocalDate), `dueDate` (LocalDate), `bookId` (String or int), and a reference to the `Student` who borrowed the book (hold the actual object, the same reasoning as the Customer reference in electric bill management).

Card management methods

- Put these on a manager class (e.g. `BorrowCardManager`) holding a `List<BorrowCard>`:
- Add: append a new `BorrowCard` to the list. Reject a `slipId` that duplicates an existing card's, since slip id is what removal looks records up by.
- Remove, by slip id: find the card with that slip id and delete it. This is also how a book return is modeled here — the exercise doesn't ask for a separate "returned" flag or date, so returning a book means removing its borrow card via this same operation.
- Display: print every current card's information, including the borrowing student's details.

Exercise 8 — Cafe Management System

Product class

- Fields: `id` (int), `name` (String), `price` (float).

Booking class

- Fields: `id` (int), `productId` (int), `productName` (String, max 20 characters), `price` (float), `date` (String), `quantity` (int).
- `id` is generated by the program itself (e.g. an incrementing counter) — the customer only ever supplies a product id and a quantity when placing a booking, never a booking id.
- `productName` and `price` are copied from the matching Product at the moment the booking is placed, not looked up again later — that way a booking still shows what the customer actually paid even if the product's price changes afterward. If the source product's name is longer than 20 characters, truncate it to the first 20 characters when copying it in.
- `date` format: use `ddmmyy` consistently (e.g. `190826` for August 19, 2026) — six digits, no separators. (The original spec writes this two slightly different ways in two places; this is the cleaner of the two and is what the search feature below also expects.)

Menu (shown once when the program starts, then again after every option):

```
Cafe Management Program
1. Enter Product | 2. Place Booking | 3. Find Booking by Date (ddmmyy)
4. Product List | 5. Booking History | 6. Exit
```

Option 1 — Enter Product

- Ask for the total number of products to enter, then collect each one's information.
- Validate: `0 <= price <= 100,000`; re-prompt on failure.
- Add each validated product straight to the in-memory product list as it's entered.

Option 2 — Place Booking

- Ask for a product id and a quantity. The id must match an existing product.
- Build a new Booking (auto-generated id, the matched product's id/name/price, today's date in ddmmyy, and the entered quantity) and add it to the booking list.

Option 3 — Find Booking by Date

- Ask for a date in ddmmyy format, then show every booking whose date matches exactly, or a "No bookings found" message if none do.

Option 4 — Product List

- Display every product, sorted primarily by name (alphabetically) and, for products that share the same name, by price as a tiebreaker.

Option 5 — Booking History

- Display every booking that's been placed, in the order they were made.

Option 6

- Exit the program.
